

1. For this assignment, I used the free version of ChatGPT (GPT-5.6 Luna) that you would find on a completely new tab of the ChatGPT website, and I used Gemini's 3.6 Flash.
2. For both LLMs, I only sent one request, which consisted of the assignment requirements directly copied from the provided instructions to this assignment, prefaced with "Please follow these instructions using C++:"
3. The code from each LLM will be below
Start of ChatGPT's code

```
#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>
#include <vector>

using namespace std;

// -----
// Email class
// -----
class Email {
private:
    string sender;
    string subject;
    string date;

    int priority() const {
        if (sender == "Boss")
            return 5;
        else if (sender == "Subordinate")
            return 4;
        else if (sender == "Peer")
            return 3;
        else if (sender == "ImportantPerson")
            return 2;
        else
            return 1; // OtherPerson
    }

    // Converts MM-DD-YYYY into a number that can be compared.
    int dateValue() const {
```

```

        int month, day, year;
        char dash1, dash2;

        stringstream ss(date);
        ss >> month >> dash1 >> day >> dash2 >> year;

        return year * 10000 + month * 100 + day;
    }

public:
    Email() {}

    Email(string sender, string subject, string date) {
        this->sender = sender;
        this->subject = subject;
        this->date = date;
    }

    string getSender() const {
        return sender;
    }

    string getSubject() const {
        return subject;
    }

    string getDate() const {
        return date;
    }

    // Returns true if this email has higher priority than other.
    bool higherPriorityThan(const Email& other) const {
        if (priority() != other.priority()) {
            return priority() > other.priority();
        }

        // Same sender category: newest email gets priority.
        return dateValue() > other.dateValue();
    }
};

```

```

// -----
// List-based MaxHeap
// -----

class MaxHeap {
private:
    vector<Email> heap;

    int parent(int index) {
        return (index - 1) / 2;
    }

    int leftChild(int index) {
        return 2 * index + 1;
    }

    int rightChild(int index) {
        return 2 * index + 2;
    }

    void swapEmails(int first, int second) {
        Email temp = heap[first];
        heap[first] = heap[second];
        heap[second] = temp;
    }

    // Moves an element upward until the MaxHeap property is restored.
    void heapifyUp(int index) {
        while (index > 0) {
            int p = parent(index);

            if (heap[index].higherPriorityThan(heap[p])) {
                swapEmails(index, p);
                index = p;
            }
            else {
                break;
            }
        }
    }
}

```

```

// Moves an element downward until the MaxHeap property is restored.
void heapifyDown(int index) {
    while (true) {
        int left = leftChild(index);
        int right = rightChild(index);
        int largest = index;

        if (left < heap.size() &&
            heap[left].higherPriorityThan(heap[largest])) {
            largest = left;
        }

        if (right < heap.size() &&
            heap[right].higherPriorityThan(heap[largest])) {
            largest = right;
        }

        if (largest != index) {
            swapEmails(index, largest);
            index = largest;
        }
        else {
            break;
        }
    }
}

public:
    // Add an email to the MaxHeap.
    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    // Return the highest-priority email without removing it.
    Email getMax() const {
        return heap[0];
    }

```

```

// Remove the highest-priority email.
void removeMax() {
    if (heap.empty()) {
        return;
    }

    if (heap.size() == 1) {
        heap.pop_back();
        return;
    }

    heap[0] = heap.back();
    heap.pop_back();

    heapifyDown(0);
}

bool isEmpty() const {
    return heap.empty();
}

int size() const {
    return heap.size();
}
};

// -----
// Main program
// -----

int main() {
    MaxHeap emailQueue;

    string line;

    // Used to remember the email displayed by the most recent NEXT.
    // This is important because two NEXT commands in a row should
    // display the same email.
    bool hasCurrentEmail = false;
    Email currentEmail;

```

```

while (getline(cin, line)) {

    // -----
    // EMAIL command
    // -----

    if (line.substr(0, 5) == "EMAIL") {
        string data = line.substr(6); // Skip "EMAIL "

        // Find the commas separating the three fields.
        size_t comma1 = data.find(',');
        size_t comma2 = data.find(',', comma1 + 1);

        string sender = data.substr(0, comma1);
        string subject = data.substr(
            comma1 + 1,
            comma2 - comma1 - 1
        );
        string date = data.substr(comma2 + 1);

        Email email(sender, subject, date);
        emailQueue.insert(email);
    }

    // -----
    // NEXT command
    // -----

    else if (line == "NEXT") {

        if (emailQueue.isEmpty()) {
            // There is no email to display.
            continue;
        }

        // Only get the highest-priority email.
        // Do NOT remove it.
        currentEmail = emailQueue.getMax();
        hasCurrentEmail = true;

        cout << "Next email:" << endl;
        cout << "Sender: " << currentEmail.getSender() << endl;
    }
}

```

```

        cout << "Subject: " << currentEmail.getSubject() << endl;
        cout << "Date: " << currentEmail.getDate() << endl;
        cout << endl;
    }

    // -----
    // READ command
    // -----
    else if (line == "READ") {

        if (!emailQueue.isEmpty()) {
            // READ removes the highest-priority email,
            // whether or not NEXT was called first.
            emailQueue.removeMax();
        }

        hasCurrentEmail = false;
    }

    // -----
    // COUNT command
    // -----
    else if (line == "COUNT") {
        cout << "There are "
              << emailQueue.size()
              << " emails to read."
              << endl;
    }
}

return 0;
}

    End of ChatGPT's code,
    Start of Gemini's Code
#include <iostream>
#include <string>
#include <sstream>
#include <vector>

struct Email {

```

```

std::string category;
std::string subject;
std::string date; // MM-DD-YYYY

// Helper to get priority level based on category (higher number =
higher priority)
int getCategoryPriority() const {
    if (category == "Boss") return 5;
    if (category == "Subordinate") return 4;
    if (category == "Peer") return 3;
    if (category == "ImportantPerson") return 2;
    if (category == "OtherPerson") return 1;
    return 0;
}

// Converts MM-DD-YYYY to YYYYMMDD integer for fast chronological
comparison
int getDateKey() const {
    if (date.length() < 10) return 0;
    int month = std::stoi(date.substr(0, 2));
    int day = std::stoi(date.substr(3, 2));
    int year = std::stoi(date.substr(6, 4));
    return year * 10000 + month * 100 + day;
}

// Overload strict total ordering: greater operator for MaxHeap
bool operator>(const Email& other) const {
    int prio1 = getCategoryPriority();
    int prio2 = other.getCategoryPriority();

    if (prio1 != prio2) {
        return prio1 > prio2;
    }

    // Newer emails come first if categories are equal
    return getDateKey() > other.getDateKey();
}
};

// List-based MaxHeap implementation built from scratch

```

```

class MaxHeap {
private:
    std::vector<Email> heap;

    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;
            if (heap[index] > heap[parent]) {
                std::swap(heap[index], heap[parent]);
                index = parent;
            } else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        int size = static_cast<int>(heap.size());
        while (true) {
            int left = 2 * index + 1;
            int right = 2 * index + 2;
            int largest = index;

            if (left < size && heap[left] > heap[largest]) {
                largest = left;
            }
            if (right < size && heap[right] > heap[largest]) {
                largest = right;
            }

            if (largest != index) {
                std::swap(heap[index], heap[largest]);
                index = largest;
            } else {
                break;
            }
        }
    }

public:

```

```

bool empty() const {
    return heap.empty();
}

size_t size() const {
    return heap.size();
}

void push(const Email& email) {
    heap.push_back(email);
    heapifyUp(static_cast<int>(heap.size()) - 1);
}

const Email& peekMax() const {
    return heap[0];
}

void popMax() {
    if (empty()) return;
    heap[0] = heap.back();
    heap.pop_back();
    if (!empty()) {
        heapifyDown(0);
    }
}
};

```

```

int main() {
    MaxHeap priorityQueue;
    std::string line;

    while (std::getline(std::cin, line)) {
        if (line.empty()) continue;

        if (line.rfind("EMAIL ", 0) == 0) {
            std::string payload = line.substr(6);
            std::stringstream ss(payload);

            Email email;
            std::getline(ss, email.category, ',');

```

```

std::getline(ss, email.subject, ',');
std::getline(ss, email.date);

// Trim leading/trailing spaces if any exist in parsing
priorityQueue.push(email);

} else if (line == "COUNT") {
    std::cout << "There are " << priorityQueue.size() << " emails
to read.\n";

} else if (line == "NEXT") {
    if (!priorityQueue.empty()) {
        const Email& nextEmail = priorityQueue.peekMax();
        std::cout << "Next email:\n";
        std::cout << "Sender: " << nextEmail.category << "\n";
        std::cout << "Subject: " << nextEmail.subject << "\n";
        std::cout << "Date: " << nextEmail.date << "\n";
    }

} else if (line == "READ") {
    if (!priorityQueue.empty()) {
        priorityQueue.popMax();
    }
}

}

return 0;
}

```

End of Gemini's code

0. Comparing

- Correctness
 - Out of the box, while the expected results and the actual results for both LLM's take on the assignment were technically correct, they were not formatted in the right way. Despite my slight disdain towards LLMs, this is likely just because of the way I pasted the guidelines for the program. Gemini's version included zero additional whitespace, and ChatGPT included empty lines after Next Email segments.

- When I stress tested edgecases, using a few custom input files, I saw that ChatGPT's version was less prone to bugs or unexpected results. Importantly, Gemini's was more prone to data parsing errors, and even crashed with a custom test file.

Execution Time

- Both versions have the same overall execution time because they both use a MaxHeap. The second version may be slightly faster in some cases because it handles certain operations more directly, but the difference is small.

Space Usage

- Both versions use $O(n)$ space because the MaxHeap stores all emails in memory. Their space usage is therefore very similar, with neither version having a significant advantage over the other.

Maintainability

- The ChatGPT's version is slightly easier to maintain because it is more clearly organized and includes more comments explaining how each part works. Gemini's version is shorter and more streamlined, but it makes more assumptions about the input format. Overall, ChatGPT's would be easier to modify and troubleshoot later.
 - Similar to the previous assignment, ChatGPT's version uses more cleanly organized commenting compared to Gemini's.
4. I decided to move forward with the code that ChatGPT generated, because the way it handled input was far more forgiving of weird cases, which I feel is pretty important. Additionally, I also think it is far more maintainable compared to Gemini's, which is also really useful.
 5. Changes go here,
 - a. Renamed sender to senderCategory for clarity.
 - b. Added dateKey to store the converted date once instead of recalculating it during every comparison.
 - c. Replaced dateValue() with convertDateToKey().
 - d. Updated the constructor to initialize dateKey.
 - e. Updated higherPriorityThan() to compare the stored dateKey.
 - f. Removed the unnecessary hasCurrentEmail variable.

- g. Changed NEXT to use `const Email& currentEmail` instead of copying the email.
- h. Changed `getMax()` to return `const Email&` instead of an Email copy.
- i. Replaced the manual email swap with `std::swap` only if you chose to make that change; your current code still uses the original manual swap.
- j. Removed the unused `<iomanip>` include.
- k. Added handling for blank input lines with `if (line.empty()) continue;`.
- l. Adjusted COUNT/NEXT output formatting to control blank lines between sections.